

LAB12: Natural Language Processing: Text Processing & Sentiment Classification

This lab will help you experience **Natural Language Processing (NLP)** by building a tiny **sentiment analysis** system.

Lab Objectives:

By the end of this lab, students will be able to:

- Understand what Natural Language Processing (NLP) is
- Perform basic text cleaning
- Convert text into numerical form (Bag of Words)
- Use a simple NLP model to classify text as Positive or Negative
- Visualize results and observe how changing text affects predictions

Introduction to NLP

NLP is the field of Artificial Intelligence that helps computers understand human language (English, Urdu, etc.). Example uses in daily life:

- Google Search
- ChatGPT
- Auto-Correct / Spell Check
- YouTube subtitles
- Sentiment detection ("positive" or "negative" reviews)

Step 0 – (Optional) Install Libraries

Run this cell **only if** you get an error that `nltk` or `sklearn` is missing.

```
# Optional: install libraries if needed (uncomment if you get ImportError)
# !pip install nltk scikit-learn
```

Step 1 – Create a Small Text Dataset

We will create dataset of sentences and their labels:

- `1` = Positive
- `0` = Negative

```
texts = [
    "I love this movie!",
    "This product is terrible",
    "Amazing experience",
    "I hate this!",
    "Very good service",
    "Worst food ever"
]

# 1 = Positive, 0 = Negative
labels = [1, 0, 1, 0, 1, 0]
```

✓ Step 2 – Clean the Text

We will:

1. Convert everything to **lowercase**
2. Remove punctuation and special characters

This makes the text easier for the computer to handle.

```
import re

def clean_text(text):
    # 1) convert to lowercase
    text = text.lower()
    # 2) keep only letters and spaces
    text = re.sub(r"[^a-zA-Z ]", "", text)
    return text

cleaned = [clean_text(t) for t in texts]
cleaned
```

```
['i love this movie',
 'this product is terrible',
 'amazing experience',
 'i hate this',
 'very good service',
 'worst food ever']
```

✓ Step 3 – Convert Text to Numbers (Bag of Words)

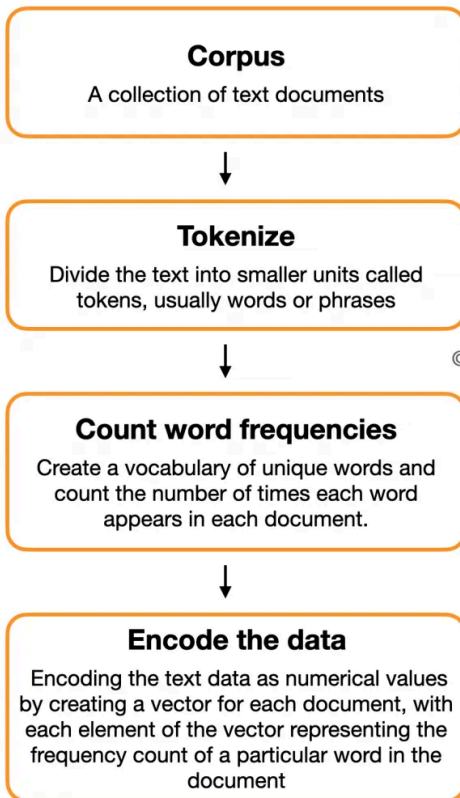
We use **CountVectorizer** to turn each sentence into a row of numbers. Each column represents a word, and the value is how many times that word appears.

vocabulary is the set of unique words or tokens extracted from a training dataset

Bag of Words (BoW) in NLP is a method for text representation that converts text into numerical vectors by counting the frequency of each word in a document, while

ignoring grammar and word order.

Bag of Words Model explanation



© AIML.com Research

Example

Corpus:
The dog is happy. The child makes the dog happy. The dog makes the child happy

Tokenization:
D1: [The] [dog] [is] [happy]
D2: [The] [child] [makes] [the] [dog] [happy]
D3: [The] [dog] [makes] [the] [child] [happy]

Documents	Counting word frequencies
D1	the: 1, dog: 1, is: 1, happy: 1
D2	the: 2, dog: 1, makes: 1, child: 1, happy: 1
D3	the: 2, child: 1, makes: 1, dog: 1, happy: 1

Encode	child	dog	happy	is	makes	the	BoW Vector representations
D1	0	1	1	1	0	1	[0,1,1,1,0,1]
D2	1	1	1	0	1	2	[1,1,1,0,1,2]
D3	1	1	1	0	1	2	[1,1,1,0,1,2]

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
vectorizer = CountVectorizer()  
X = vectorizer.fit_transform(cleaned)
```

```
print("Vocabulary (words the model knows):")  
print(vectorizer.get_feature_names_out())
```

```
print("\nBag of Words representation (as array):")  
X.toarray()
```

```
Vocabulary (words the model knows):  
['amazing' 'ever' 'experience' 'food' 'good' 'hate' 'is' 'love' 'movie'  
 'product' 'service' 'terrible' 'this' 'very' 'worst']
```

```
Bag of Words representation (as array):  
array([[0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0],  
       [0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0],  
       [1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],  
       [0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0],  
       [0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0],  
       [0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]])
```

```
# Create a table for Vocabulary and Bag of Words representation  
import pandas as pd
```

```
# Convert sparse matrix to dense array
```

```
bow_array = X.toarray()

# Get the vocabulary (words)
words = vectorizer.get_feature_names_out()

# Create a DataFrame
df_bow = pd.DataFrame(bow_array, columns=words)

# Display the table
df_bow
```

	amazing	ever	experience	food	good	hate	is	love	movie	product	serv
0	0	0	0	0	0	0	0	1	1	0	
1	0	0	0	0	0	0	1	0	0	1	
2	1	0	1	0	0	0	0	0	0	0	
3	0	0	0	0	0	1	0	0	0	0	
4	0	0	0	0	1	0	0	0	0	0	
5	0	1	0	1	0	0	0	0	0	0	

✓ Step 4 – Train a Simple Sentiment Classifier

We use **Logistic Regression** (a simple classification algorithm) to learn which words are linked to positive or negative sentences.

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X, labels)

print("Model trained!")

Model trained!
```

✓ Step 5 – Test With Your Own Sentence

Change the text in `user_input` and run the cell to see if the model thinks it is **Positive** or **Negative**.

```
def predict_sentiment(sentence):
    # clean the sentence like training data
    cleaned_sentence = clean_text(sentence)
    # convert to bag of words using the same vectorizer
    vec = vectorizer.transform([cleaned_sentence])
```

```
# make prediction
pred = model.predict(vec)[0]
# show result
print("Cleaned sentence:", cleaned_sentence)
if pred == 1:
    print("Prediction: Positive Sentence")
else:
    print("Prediction: Negative Sentence")
```

```
# Try your own sentence here
user_input = "I love to watch movies!"
predict_sentiment(user_input)
```

```
Cleaned sentence: i love to watch movies
Prediction: Positive Sentence
```

✓ Lab Submission Tasks / Experiments

Try these experiments and observe how predictions change:

Task 1 – Try Different Sentences

Change `user_input` to:

- `"This is amazing"`
- `"I don't like this"`
- `"Worst experience of my life"`
- `"Fantastic food and great service"`

Run the prediction cell each time and note the outputs.

Task 2 – Add More Training Data

Add new sentences and labels, for example:

```
texts.append("I am so happy today")
labels.append(1)

texts.append("This is so bad")
labels.append(0)
```

Then re-run:

1. The **cleaning cell** (Step 2)
2. The **Bag of Words cell** (Step 3)
3. The **training cell** (Step 4)

Test again with your own sentences. Did the model improve?

✓ Task 3 – Reflection Questions

Answer in your own words:

1. What is **NLP**?
2. Why do we **clean** the text before training?
3. What is a **Bag of Words** model?
4. Why does the computer need **numbers instead of raw text**?
5. What is **sentiment analysis** used for in real life? (Give at least 2 examples.)

Double-click (or enter) to edit